Page Matrix — Technical Reference

Engineering-oriented reference for the **dashboard**: how it scans pages, how it scores them, how it rolls them up, the site checks, and the Phase 5 UI. Since Phase 3 the dashboard uses **the same check engine as the panel** — the old 5-pillar model (`site-score.ts`: Crawlability, Structured, Answerability, Authority, Retrievability) is gone. The checks themselves (IDs, thresholds, credit) are documented once, in `PAGE-CHECKS-TECHNICAL.md`.

Source of truth:

- `src/hooks/useSitePages.ts` — page enumeration, scanning, the score cache, site-check inputs
- `src/lib/checks/` — the engine (see `PAGE-CHECKS-TECHNICAL.md`)
- `src/lib/checks/view-model.ts` — `PageScoreSummary`, `CATEGORY_META`, Fix first grouping, status words
- `src/lib/site-aggregate.ts` — site roll-up
- `src/app/dashboard-widget-extension/page.tsx` and `src/components/dashboard/*.tsx` — the UI (one component per tab)
- `src/hooks/useScoreHistory.ts` — the last 8 site scores
- `src/hooks/useSiteType.ts` — the Site type setting
- `src/types/dashboard.ts` — `PageMatrixRow`, `MatrixFilter`

Scoring model

Each page is scored by `runPageChecks` with a context from `buildPageContext` — the same function the panel uses — and summarised by `summarisePageReport` into a `PageScoreSummary`:

Field	Meaning
`score`	0–100, or `null` when the gate blocks the page or nothing could be scored
`label`	`not-ready` / `almost-there` / `agent-ready` from `labelForPct` (80 / 60), or `null`
`blocked`, `gateReason`	the gate (P1): Sitecore Robots Meta Tag field, or a rendered noindex / nosnippet / max-snippet:0
`categories`	earned and max points for `reach`, `read`, `understand`, `trust` (max 0 when nothing in that category was scored)
`issues`	every scored check that isn't passing: `{ id, status, gain }` with `status` "warning" or "fail" and `gain` = `maxPoints − points`. Empty when blocked. Feeds Fix first

Pools: Essential checks share 80 points, Recommended 20, Bonus adds up to 5, capped at 100. N/A and not-checked results are left out of the pools. Full detail in `PAGE-CHECKS-TECHNICAL.md`.

Rating bands — `labelForPct`, `src/lib/score-thresholds.ts`

Label	Range	Shown as (`LABEL_TEXT`)
`agent-ready`	≥ 80	Agent-ready
`almost-there`	60–79	Almost there
`not-ready`	< 60	Not ready

The same `READY_PCT` (80) and `ALMOST_PCT` (60) drive the score rings, the heatmap cells, the verdicts and the site roll-up.

Scanning a page (`scanOne`, `useSitePages.ts`)

- **Context first:** `application.context` is read before any XM Cloud call, and its Context ID (`resourceAccess[].context.preview`, else `.live`) is sent as `sitecoreContextId` on `xmc.sites.retrieveSite` and every authoring GraphQL call. The console line `[sitePages] application.context →` shows the Marketplace app type (`custom` / `public`), the extension points (for dashboard blocks, each widget's route is under `blocks`, and the top-level route is null), and whether a Context ID was found. It names fields one by one, because the raw context can hold the editing secret. `[sitePages] host.state →` shows which Sitecore host runs the widget: `isNull: true` for the Cloud Portal, `environment` and `language` for SitecoreAI.

- **Host refusing authorization:** if the Sitecore host refuses a call with "getAccessToken method not provided" (plan section 13, item 1), `isHostAuthError` (`src/lib/xmc-errors.ts`) recognises it. The Overview then says Sitecore didn't authorize the request, instead of falling back to the content-tree walk and reporting a missing Home item. Other errors, including a plain 401 from an API, keep the existing fallbacks.
- **Enumeration:** authoring GraphQL walks the site's Home item 5 levels deep; items with `__Renderings` or `__Final Renderings` are pages. For each it reads `template { name }`, `__Updated`, and the fields the checks read, by the shared names in `SCORING_FIELD_NAMES` (`pageTitle`, `MetaDescription` — overridable with `NEXT_PUBLIC_FIELD_TITLE` / `NEXT_PUBLIC_FIELD_META_DESCRIPTION` — and `robotsMetaTag`). `pageFieldsFromValues` normalises them into `row.fields`, exactly as the panel does. The OG description is also read, only as the llms.txt summary fallback.
- **Scan:** `/api/preview-fetch` for the live HTML and `/api/markdown-check` for P5 and P6 (the page requested as an agent and as a browser: 2 more requests per page, in parallel with preview-fetch; a failed request leaves P5 "not checked"; see PAGE-CHECKS-TECHNICAL, "P5 and P6 in detail"), then `/api/ai-check` with `ParsedHtml.mainText` (on `ANTHROPIC_MODEL_BULK`, cached by content hash in localStorage, shared with the panel, and on the server). PageSpeed doesn't run in the dashboard, so P7 is "not checked" and left out of the score. The dashboard is **live-only**: it has no Page Builder draft (`getPageHTML` works only in Page Builder), so it passes no `draft` to `buildPageContext`, and P3 is "not checked" ("Page Builder only") and left out too. The same inputs give the panel's score; the panel's can differ when it scores a draft (PAGE-CHECKS-TECHNICAL, "Draft and live"). `mainText` now includes collapsed FAQ answers, so accordion pages get a new AI cache key once.
- **Concurrency:** 5 pages at a time; all unscanned rows scan in the background after enumeration.

Score cache

Summaries are cached in localStorage under `aiDiscovery:scores:v6:<siteId>`, keyed by page URL and reused while the page's `__Updated` is unchanged. **v6** (Phase 7) adds P5 and P6 and `servesMarkdown` (P5 scored with any credit → true, with none → false, not checked → null); **v5** (Phase 5) added `issues` and the new label names; **v4** was the unified engine. A cached score doesn't follow a site-wide Markdown change, since `__Updated` doesn't change: **Scan site** after the developer turns Markdown on. A new version makes the dashboard rescan every page once. The AI results have their own content-hash cache, so pages whose text hasn't changed don't call `ai-check` again. **Scan site** clears the cache and re-fetches the site-check inputs.

Site roll-up (`site-aggregate.ts`)

- **Overall score:** mean of the page scores. Unscanned and **blocked** pages are left out; blocked pages are counted separately ("N Blocked" in the summary).

- **Category cards:** for each category, the mean of `earned ÷ max` over pages where that category was scored. A page "needs work" in a category below 60% (`FAIL_THRESHOLD` , the Almost there line; it was 40% before Phase 5). The "What's included" list comes from the check registry (`pageCheckTitles`), so it always matches what's scored; P3 and P7 are marked "(Page Builder panel only)". The **Read** card shows "N of M pages not served as Markdown" in red instead of its "needs work" line when any scored page doesn't serve Markdown (`SiteAggregate.markdown` : `served` and `checked` over the scored pages, from `servesMarkdown`), as in the mockup.
- **Pages heatmap:** a category cell shows `round(earned ÷ max × 100)` , coloured by `labelForPct` , or N/A when nothing in it was scored. Blocked pages show the gate reason across the cells and sort with the unscanned ones.

Fix first (`view-model.ts`)

- **Page issues:** `groupPageIssues` groups every page's `issues` by check ID. `gainEach` is the mean gain, rounded ("+N pts each"); groups under 1 point are dropped. The worst status wins (any `fail` → Fix). Sorted by the total gain across pages.
- **Site issues:** `siteIssueGroups` turns the site checks that need work into groups that sort first ("Affects every page").
- **Markdown once** (`fixFirstGroups`): S3 and P5 are the same developer fix, so once pages have been checked, P5's page group (its page count, "+N pts each" and the list of pages in the note) replaces S3's site row and comes right after the site issues. Before any page has been checked, S3's row shows.
- **Buttons:** site issues and Developer page issues get **Send to developer**, which copies `issueNote` (the issue, why, and each page's URL; for a site issue, `developerNote` with its fix and snippet). Author page issues get **Review pages**, which opens the Pages tab filtered to `pageIds` . Bulk "Generate for all" is Phase 9.
- The hero sentence comes from the same groups: the number of site issues, or the biggest page issue, and how many issues authors can fix.

History (`useScoreHistory.ts`)

`useSiteHistory` keeps the last 8 site scores in localStorage (`aiDiscovery:siteHistory:v1:<siteId>`). It adds one when a scan finishes in this session (pages were scanning, now none are), not on every load from cache. The hero shows "▲ N since last scan" (the last two entries) and the trend line from two scans on. Phase 8 decides on shared storage for "What changed".

Agent access tab: site checks

All twelve site checks (S1–S12, Phase 6) grade one scan: `GET /api/agent-access?domain=<domain>` (`src/app/api/agent-access/route.ts`, collector in `src/lib/server/agent-access.ts`). It returns `AgentAccessData` (`src/lib/agent-access/types.ts`): raw evidence only, no grades. The checks in `src/lib/checks/site/` grade it synchronously, so the engine stays pure and the panel, the dashboard and the demo fixture all go through the same code. The check definitions and their credit are in `PAGE-CHECKS-TECHNICAL.md` ("Site checks").

The scan. Every request goes through `safeFetch`. The bot and browser requests send their User-Agent and Accept as `publicHeaders`, which `safeFetch` keeps across a cross-origin redirect (example.com → www.example.com), so a redirect doesn't silently turn a bot request into a default one.

- **Round 1:** robots.txt, the home page as a browser (Chrome's User-Agent), the home page with `Accept: text/markdown`, `/llms.txt`, a random path (`/agent-ready-check-<hex>`), and 11 well-known paths (`src/lib/agent-access/well-known.ts`).
- **Round 2**, once robots.txt and the home page are in: two sample pages (the first internal page links on the home page, links in `<main>` first, no files), each bot × (home + samples), the sitemap (robots.txt's `Sitemap:` line on the site's own origin, else its first `Sitemap:` line, else `/sitemap.xml`; for an index, the first child sitemap), and the about, contact and privacy pages linked from the home page (footer first, shortest path).
- **Bots** (`src/lib/agent-access/bots.ts`): 12, grouped Search / User / Training. Google-Extended and Applebot-Extended are robots.txt tokens only, so they're never requested. A bot that robots.txt disallows on `/` isn't requested either.
- **Budget:** at most 6 requests at a time (a priority queue: robots.txt and the home page, then the bot requests, then the rest), 6 s per request, and a 17 s deadline for the whole scan. Requests not finished by then come back with `error: "deadline"`. The route sets `maxDuration = 30`. About 50 requests for a site that allows the agents; BrandX (2026-11-02), whose robots.txt disallows everyone, took 22 requests in 1–3 s; its Vercel replica, which lets in four of the bots, took 34 requests (12 of them as bots) in 1.4–3.1 s.
- **Bad input:** a private or invalid domain → 400 `blocked_url` (a hostname that resolves to a private address is caught at connect time, on the home request).

The grid (`src/lib/agent-access/grid.ts`, `botGrid`) turns the evidence into one row per bot: robots.txt verdict for `/` (RFC 9309 matching, `src/lib/agent-access/robots.ts`), the live column (`200`, `403`, `200 · 403 on /insights`, `Challenge page`, `Not requested`, `robots.txt rule only`) and the result. Each bot page is compared with the browser's request for the same page: a 4xx/5xx is **blocked**; a 2xx with challenge markers the browser didn't get, or under 20% of the browser's size (when the browser's page is at least 5 KB), is a **challenge page**; a failed request is **Couldn't check**; a page the browser couldn't load either isn't compared. The worst page wins. A blocked training agent is "your choice" (info), never a fail. S1 and S2 grade these same rows, so the grid and the checklist can't disagree.

Fetching and caching (`src/hooks/useAgentAccess.ts`): once per domain, kept in localStorage for an hour (`aiDiscovery:agentAccess:v1:<domain>`), shared with the panel. **Scan site** calls `refresh` , which skips the cache. The dashboard runs `runSiteChecks` with the scan, the home page HTML it fetched (for S7), the organisation for the snippet (`getOrganizationInfo`) and the expected one (`getConfiguredOrganization`), and the site type. `useSitePages` no longer fetches robots.txt or the home page. Site checks never feed page scores.

Agent access tab (`AgentAccessTab.tsx`): the checklist (status word, title, tier, message, and a button: S2 → **Open robots.txt help**, S5 → **Open generator**, S9 when not set → **Generate line**, all opening Files; S7 → **Copy snippet**; S3 → **See fixes**, which opens the Agent View's fix cards (`MarkdownFixes`) and **Send to developer** under the row; other developer checks → **Send to developer**; S6 is the author's, so no button). S3's message adds "Markdown on N of M scanned pages" (`SiteContext.markdownPages` from the page roll-up); its credit stays the home page's. A scored Bonus result with no credit shows **Not set** instead of N/A. Beside it, the bot grid with an All / Search / User / Training filter, a red box when search or user agents are blocked by the firewall (with **Send to developer**, which copies S1's developer note), the caveat that requests come from our server, and "Checked N ago · N requests in N s". While the scan runs: "Requesting your pages as each AI agent. This takes up to 20 seconds."; on failure, **Try again**.

Files tab (`FilesTab.tsx`): the llms.txt generator (`buildLlmsTxt` , `whenToUse` → `## When to use this site` , the 60 threshold) with a ✓ **Published / ✕ Not published** chip from the scan; and the robots.txt helper: checkboxes for the search, user and training groups and the Content-Signal line, which start from what robots.txt allows today (the card re-mounts per scan). `buildRobotsLines` (`src/lib/agent-access/robots-helper.ts`) writes a group per purpose with `Allow: /` or `Disallow: /` , then `User-agent: *` / `Content-Signal: search=…, ai-input=…, ai-train=…` / `Allow: /` , then the site's own `Sitemap:` URL from robots.txt (`siteSitemap` , as in the scan; or `/sitemap.xml`). **Send to developer** adds a note to keep the existing rules.

Site type (Content site / Commerce / App or API) is a select in the header, stored per site in localStorage (`aiDiscovery:siteType:v1:<siteId>`). S11 applies only to App or API, S12 only to Commerce; otherwise they're N/A ("Only for App or API sites. Change the site type to include it."). The well-known paths are fetched for every type, so changing the type needs no rescan.

What changed from the pillar model

Old pillar signal	Now
Crawlability: AI crawler permissions	S2 `reach.robots-ai-rules` (site check, bot groups by purpose since Phase 6) and S1 `reach.ai-bot-access` (live requests)
Crawlability: canonical	P2 `reach.canonical` (Essential)
Crawlability: robots meta	P1 gate — blocks the score instead of costing 4 points
Structured: FAQPage / Article / BreadcrumbList	P10 `understand.json-ld` — now validates required properties, reads `@graph` and `@type` arrays, expects FAQPage from real Q&A pairs instead of question H2s
Structured: Organization (home page)	S7 `understand.org-schema` (site check)
Answerability: headings	P8 `understand.headings`
Answerability: question H2s	P13 `understand.question-headings`
Answerability: reading grade + paragraph length	P14 `understand.readability`
Authority: AI uniqueness	P16 `trust.uniqueness`
Authority: content depth	P19 `trust.depth`
Authority: freshness	P18 `trust.freshness` (adds JSON-LD `dateModified`; reads Sitecore's compact date format)
Authority: descriptive anchors	P20 `trust.anchor-text` (prefix rule narrowed, SPIKE-FINDINGS §2)
Retrievability: title + description	P9 `understand.title-meta` (Sitecore field first; title band now 50–60)
Retrievability: OG tags	P11 `understand.metadata` (adds `html lang`, `og:type`)
Retrievability: landmarks	P4 `read.landmarks`
Retrievability: image alt	P15 `understand.image-alt` (N/A without images)

New in the dashboard because it now shares the panel's checks: P11's lang and og:type, P12 FAQ answer quality, P17 author (article templates), P21 follow-up gaps (Bonus).

UI (Phase 5)

Blok tokens and components (`src/components/ui`), `@mdi/js` icons, and the shared components in `src/components/shared` (also used by the panel; see PAGE-CHECKS-TECHNICAL). Tabs are Radix tabs: arrow keys move between them. A URL hash (`#pages` , `#agent-access` , `#citations` , `#files`) opens that tab; it's read after mount so hydration matches. Header: site name and domain, Site type, "Last scan N ago" (the newest `lastScannedAt`), and **Scan site** (`clearAndRefresh`). The page-list error shows on Overview and Pages with **Retry**; the other tabs only need the domain.

Demo mode (`?demo=1`)

`makeTestSitePages()` (`src/lib/rule-engine/test-fixture.ts`) builds 12 rows by running the **real engine** on variants of the panel's demo page, so the numbers are consistent. Every row also gets `makeTestMarkdownCheck` : by default the page returns HTML for `Accept: text/markdown` , as in the mockup (P5: Fix on every page), so the site has 5 Not ready, 4 Almost there, 2 Agent-ready and 1 blocked (noindex), a site score of 62, and the Read card says "11 of 11 pages not served as Markdown". The row "Blog: content strategy" is the panel's demo page unchanged, scored on live HTML only (74). The panel demo also scores its demo draft (`makeTestDraft` : an edited heading and client-rendered Related Articles cards), so it shows 70, with the "Draft differs" banner and a P3 warning. `?demo=1&md=1` (`useDemoServesMarkdown`) switches the panel, the rows and the agent-access fixture to a site that serves Markdown (P5 4 of 4, P6 Pass, S3 Pass): 2 Not ready, 4 Almost there, 5 Agent-ready, a site score of 79; the content-strategy row 91, the panel 84. The demo page's HTML (`makeTestPageSource` , live and draft) is real markup, so the Agent View converts it like a real page. `makeTestSiteContext()` gives the Agent access tab a BrandX-style home page with no JSON-LD (S7: 0, warning, snippet) and `makeTestAgentAccess()` , the scan from the Symposium mockup: the firewall answers ChatGPT-User and Perplexity-User with 403 although robots.txt allows them (S1: Fix, the Phase 6 accept-when case), robots.txt opts GPTBot out of training (S2: Pass with a note), HTML for `Accept: text/markdown` (S3: Fix), a 128-URL sitemap listed in robots.txt (S4: Pass), no llms.txt (S5: Improve), about, contact and privacy published (S6: Pass), a real 404 (S8: Pass), no Content-Signal line or Link header (S9, S10: Not set).

Demo mode never starts the Marketplace client (`useMarketplaceClient` checks `isDemoUrl()`), in the dashboard or the panel. So it makes no SitecoreAI calls and can't write to a real page, even if `?demo=1` is opened inside a Sitecore extension point. The panel doesn't auto-scan in demo mode either: the fixture already holds the HTML, the draft and the AI results, so nothing fetches the fixture URL, reads `getPageHTML` or calls `ai-check` on load. Agent access, Files and the panel's site-issues strip use `makeTestAgentAccess()` (`useAgentAccess` returns the fixture and never fetches), so neither the dashboard nor the panel calls `/api/agent-access` in demo mode; `useMarkdownCheck` likewise returns the panel's Markdown fixture without calling `/api/markdown-check` , and the preview is converted in the

browser. The trend line uses `DEMO_SITE_HISTORY_BEFORE` plus the current score, and **Scan site** is off. Generate meta and Citation forecast still call Claude when clicked.

Key files

Aspect	File
Enumeration, scanning, score cache	`src/hooks/useSitePages.ts`
Site checks' scan: route, collector, client cache	`src/app/api/agent-access/route.ts`, `src/lib/server/agent-access.ts`, `src/hooks/useAgentAccess.ts`
Bots, robots.txt parsing, the grid, the robots.txt helper, well-known paths	`src/lib/agent-access/`
P5/P6's requests (one page as an agent and as a browser)	`src/app/api/markdown-check/route.ts`, `src/lib/server/markdown-check.ts`, `src/lib/markdown/`
Host authorization refusal	`src/lib/xmc-errors.ts`
Demo mode check (`?demo=1` , `&md=1`)	`src/hooks/useDemoMode.ts`
Check engine, registry, scoring	`src/lib/checks/`

Aspect	File
Summary shape, category labels, Fix first grouping, developer notes	`src/lib/checks/view-model.ts`
Site roll-up	`src/lib/site-aggregate.ts`
Dashboard UI	`src/app/dashboard-widget-extension/page.tsx`, `src/components/dashboard/{OverviewTab,PagesTab,AgentAccessTab,FilesTab,Sparkline}.tsx`
Shared UI (rings, chips, status icons, buttons)	`src/components/shared/`, Blok in `src/components/ui/`
Score history	`src/hooks/useScoreHistory.ts`
llms.txt builder	`src/lib/llms-txt/build.ts`
Site type setting	`src/hooks/useSiteType.ts`
Dashboard types	`src/types/dashboard.ts`
Demo data	`src/lib/rule-engine/test-fixture.ts`